

The Complete Theory Guide

Smart Scan Strategy for Electronic Warfare — SIH Problem Statement 26055

A teach-yourself notebook: what we are building, why it works, and every idea you need before writing a single line of code.

How to use this document

This is written so that someone with **zero background in radar, electronic warfare, or machine learning** can read it start to finish and end up able to explain the entire project to a judge. Every new term is defined the first time it's used. Every formula has a worked numeric example right next to it. Nothing is assumed.

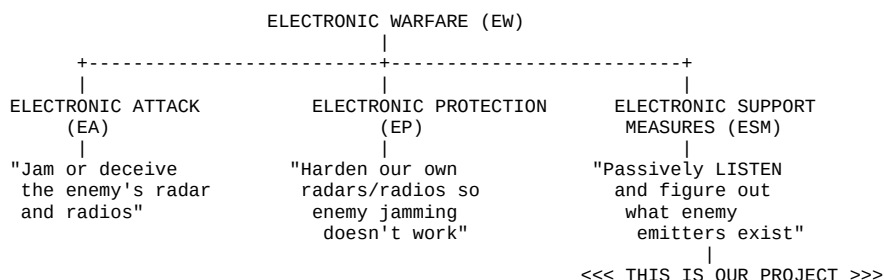
Read it in order. Don't skip Part 1 even if you think you already know what "radar" means — the whole project only makes sense once you understand the *specific* hardware limitation we are working around.

PART 1 — THE WORLD WE ARE OPERATING IN

1.1 What is Electronic Warfare (EW)?

When two militaries face each other, they don't just fight with tanks and planes. They also fight over the **invisible radio spectrum** — the range of radio frequencies that radars, radios, and jammers all use to "talk," "see," and "listen."

Electronic Warfare has three branches. Only one of them is what we are building for.



Electronic Attack (EA): Actively transmitting energy to blind or confuse an enemy radar or jam their radio.

Electronic Protection (EP): Making your own radar/radio resistant to being jammed.

Electronic Support Measures (ESM): Silently listening to the spectrum, without transmitting anything, to detect: “Is there a hostile radar out there? What frequency? What kind of radar is it? Where is it, roughly?”

Why ESM matters most here

ESM is the **first step** in the whole EW chain. You cannot jam a radar (Electronic Attack) until you’ve first detected it exists and know its frequency (ESM). You cannot fire an anti-radiation missile at an enemy radar until ESM has located it. **ESM is the eyes and ears that everything else depends on.**

And critically: **ESM never transmits.** It only listens. This means the enemy has zero way of knowing they are being watched — no radar warning receiver lights up on their end, because we never sent them a signal to detect in the first place. This is why our project is valuable: a good passive listener gives free, undetectable early warning.

1.2 The Hardware Bottleneck — Why This Problem Even Exists

The “Hotel with 50 Rooms” Story

Imagine you are a security guard outside a hotel with **50 rooms**. Somewhere inside, people are having secret conversations. Your job: catch as many of these conversations as possible.

The catch:

- You can only stand outside **one door at a time** and listen through it.
- The people inside know someone might be listening, so they **talk in short, quick bursts** — a few seconds here, a few seconds there — and often move to a **different room** to keep talking.
- Every time you walk from one door to another, it takes a few seconds — and during that walk, **you can’t hear anything at all**, from any room.

If you just check Room 1, then Room 2, then Room 3, in a fixed boring order (this is called a **round-robin sweep**), by the time you get back to Room 1, you may have already missed three separate conversations that happened and ended while you were elsewhere.

This exact problem is what our software solves — except the “rooms” are radio frequency channels, and the “conversations” are enemy radar and radio signals.

Translating the story into real RF (radio frequency) terms

Story element	Real-world equivalent
The hotel’s total floor, all 50 rooms	Surveillance Bandwidth — the whole slice of radio spectrum we must watch (could be hundreds of MHz to several GHz wide)
One door you can listen at	Instantaneous Bandwidth (IBW) — the narrow slice of spectrum your receiver hardware can actually process at any single

Story element	Real-world equivalent
	moment
Number of doors/rooms	$N = \text{Surveillance Bandwidth} \div \text{Instantaneous Bandwidth}$ — this gives you the number of distinct “channels” you must choose between
Walking time between doors (can’t hear anything)	Tuning Latency (τ_{switch}) — the real physical delay when your receiver’s internal electronics (local oscillator) re-tune to a new frequency; during this delay you are blind
Short secret conversation bursts	A radar pulse or radio transmission that only lasts a short time before it stops
People changing rooms	An enemy radar or jammer that hops to a different frequency to avoid being tracked

Why this makes it a Two-Dimensional Search Problem

You are not just trying to find the right **frequency**. You are trying to find the right frequency **at the right moment in time**. Miss the timing and it doesn’t matter that you were “listening in the right room” — you weren’t listening at the right second.

THE 2D SEARCH GRID (rows = frequency channels, columns = time slots)

Channel	Slot1	Slot2	Slot3	Slot4	Slot5	Slot6
Band 4		SIGNAL			SIGNAL	
Band 3	SIGNAL			SIGNAL		
Band 2	[YOU'RE TUNED HERE THE WHOLE TIME — MISS EVERYTHING ELSE]					
Band 1			SIGNAL			SIGNAL

If you park your receiver on Band 2 the whole time (or sweep in a fixed pattern that never happens to align with these signals), you catch ZERO of the six signals that actually occurred.

This is why the problem statement calls interception “a two-dimensional search problem” — you must match **frequency AND time** simultaneously, with a receiver that can only look at one small piece of that grid per moment.

1.3 Why “Old” Strategies Fail (Open-Loop Scanning)

An **open-loop** strategy is one that is decided **in advance**, before the mission even starts, and then never changes based on what actually happens during the mission. Think of it like a fixed train timetable that doesn’t care if the train station is actually busy or empty that day.

Strategy A: Round-Robin Sweep

Check Band 1, then Band 2, then Band 3... up to Band N, then loop back to Band 1. Forever, in that exact order, no matter what.

The Flaw: If a signal happens on Band 2 exactly while you’re sweeping through Band 45, you miss it completely — and you won’t be back at Band 2 for a long time.

The Blind Zone Problem: If an enemy radar happens to pulse at a rate that lines up with your sweep cycle (a “harmonic” relationship), you might arrive at that channel *every single time* right after the pulse has already ended — and never catch it, no matter how many times you sweep. Bad luck, structurally guaranteed to repeat.

Strategy B: Priority-Weighted Sweep

Same idea, but you spend more time on frequencies that pre-mission intelligence says are “likely dangerous” (e.g., known enemy radar bands from past intel reports).

The Flaw: Pre-mission intelligence goes stale fast in an actual conflict. The enemy might introduce a new radar type you have no intel on, or move operations to a frequency your intel didn’t flag. You end up “camping” on old, now-quiet channels while missing the real, current threat.

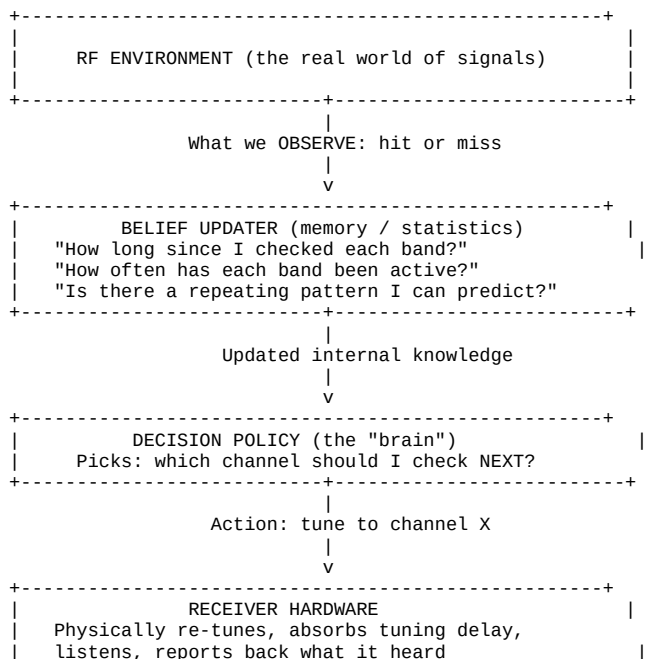
The core problem with BOTH strategies

They never learn. They don’t look at what has actually happened during the mission (hits and misses) and adjust. They are deaf to their own experience.

PART 2 — THE SOLUTION: CLOSED-LOOP, LEARNING-BASED SCANNING

2.1 What “Closed-Loop” Means

A **closed-loop** system uses its own past observations to make its next decision. It has a feedback loop:



```

+-----+
|
| (loops back to the top – repeat forever)

```

This loop repeats every “dwell” (one discrete moment of listening time). Every single loop, the system gets a little smarter about where signals tend to appear and when.

2.2 Why This is Called a “Restless Multi-Armed Bandit” (RMAB)

This is the single most important theoretical idea in the whole project. Take your time with this section.

First, what is a plain Multi-Armed Bandit (MAB)?

Imagine a row of slot machines (“one-armed bandits”) in a casino. Each machine has a different, unknown chance of paying out. You have limited pulls. You want to figure out which machines pay best, while also actually collecting winnings along the way. This is the classic **explore vs exploit** trade-off:

- **Explore:** try a machine you haven’t tried much, to learn about it.
- **Exploit:** pull the machine you already believe is the best one, to cash in.

Crucial rule in a classic bandit: a slot machine you are NOT pulling stays exactly the same. It doesn’t change while you ignore it. It’s frozen until you interact with it again.

Now, why our problem is “Restless”

In our RF world, an enemy radar does **not** freeze just because we aren’t currently listening to its frequency. It turns on. It turns off. It hops to another band. All of this happens **whether or not we are watching**. The “slot machine” keeps spinning on its own even when we’re not pulling its lever.

This is called a **Restless** Multi-Armed Bandit, because the arms (channels) are “restless” — they keep changing state on their own, driven by the enemy’s behavior, independent of our actions.

CLASSIC BANDIT	RESTLESS BANDIT (our problem)
-----	-----
Arm untouched = frozen	Arm untouched = still evolving (enemy radar can turn on/off, hop frequencies, regardless of whether we're watching)

Why we also call it “Partially Observable” (POMDP)

We only ever get information about the **one channel we chose to check** at each moment. We learn nothing directly about the other 49 channels during that same moment — we only have old, aging information about them. This is called **partial observability**: we never see the full true state of the world, only a small window into it.

Put together: our problem is a Restless, Partially Observable Multi-Armed Bandit — which is a well-studied area of research (used originally for things like Wi-Fi channel access, since 2007–2010, by researchers like Zhao & Krishnamachari). We are not inventing a brand-new

field — we are correctly applying an existing, respected mathematical framework to a defense problem.

2.3 Reinforcement Learning — The Absolute Basics

Before diving into our specific algorithms, you need the three core ideas of Reinforcement Learning (RL) in plain English.

RL term	Plain-English meaning	In our project
State (s)	“What does the agent currently know or believe about the situation?”	How long since each channel was last checked, recent hit/miss history per channel, any detected patterns
Action (a)	“What choice does the agent make?”	Which single channel to tune the receiver to next
Reward (R)	“How good or bad was that choice, as a number?”	A score given after each action — see below

Our Reward Function, explained with real numbers

We want the agent to be rewarded for: 1. Finding a **brand-new** signal it hasn’t seen before — **big reward** (this encourages exploring) 2. Catching an **already-known** active signal again — **small reward** (still useful, but less exciting than something new) 3. Checking a channel that turns out **empty** — **small penalty** (wasted time) 4. **Switching channels unnecessarily** — **small penalty** (because switching costs tuning delay — real wasted listening time)

Worked example:

Say our reward values are: $R_{\text{new}} = +10$, $R_{\text{hit}} = +2$, $C_{\text{empty}} = -1$, $C_{\text{tune}} = -0.5$

- Agent switches to a channel it’s never gotten a hit on before, and catches a signal there for the first time: $+10 - 0.5$ (switch cost) = $+9.5$
- Agent stays on the same channel it was already on, and it’s still active: $+2 - 0$ (no switch) = $+2$
- Agent switches to a channel, and it’s empty: $-1 - 0.5 = -1.5$

Over many, many repeated dwells, the agent naturally learns: “channels that have been quiet for a long time and I haven’t fully explored are worth checking (for the big reward if something new is there), but channels I already know are hot are worth revisiting too (smaller, but safer reward), and pointless empty checks or excessive switching cost me.”

PART 3 — THE ALGORITHMS: HOW THE “BRAIN” ACTUALLY DECIDES

We use several different decision-making strategies, from simplest to most advanced. We build and test them **in this order**, because each one is a safety net if the next, more complex one, isn't ready in time.

3.1 Baseline 1 — Round Robin (the “dumb” comparison point)

Explained above already. We keep this in our final system purely as **proof of improvement**. Without it, we cannot show a judge “our system is X% better” — X% better than *what*? You always need a boring baseline to measure against.

3.2 Baseline 2 — Priority-Weighted Sweep

Also explained above. Second boring baseline, slightly smarter than round-robin, still open-loop.

3.3 Upper Confidence Bound (UCB) — Our First “Smart” Algorithm

The idea, in one sentence: for every channel, calculate a score that combines “how good has this channel been so far” PLUS “how uncertain am I about this channel” — and always pick the channel with the highest combined score.

Why add the “uncertainty” part? Because a channel we've barely checked could secretly be great — we just don't have enough data yet to know. UCB deliberately gives unexplored channels a bit of a “benefit of the doubt” boost, so they get checked occasionally even if their early results look mediocre.

Simple analogy: You're picking a restaurant. Restaurant A you've been to 20 times and it's always a solid 7/10. Restaurant B you've been to once and it was a 9/10 — but that's just one visit, could be luck. UCB says: “give Restaurant B a few more tries before fully trusting that 9/10, because we're less *certain* about it.”

3.4 Thompson Sampling — An Alternative “Smart” Algorithm

The idea, in one sentence: instead of calculating a fixed score, keep a probability distribution (a range of “how likely this channel is to be active”) for each channel, randomly sample a guess from each distribution, and pick whichever channel's random sample came out highest.

This naturally balances explore/exploit too, but through randomness guided by belief, rather than a fixed formula. It tends to perform very well in practice and is simple to implement.

3.5 Whittle Index — Our Key Differentiator

This is the algorithm specifically designed for **restless** bandits (UCB and Thompson Sampling above were originally designed for the *classic*, non-restless bandit — they still work reasonably here, but aren't purpose-built for our exact problem).

The idea, in one sentence: for each channel, compute a single number (the “Whittle index”) that represents “how urgent is it to check this channel right now,” based on how long it's been

OUR NEXT ACTION:

```
[PREDICTIVE DWELL]
→ we deliberately
  check this channel
  right before t7,
  because we predict
  a pulse is due
```

4.2 Why a Normal Fourier Transform (FFT) Doesn't Work Here

A standard Fast Fourier Transform, which is the usual tool for finding repeating patterns in data, **requires evenly-spaced samples in time**. Our data is the opposite — irregular gaps, because we're only glancing at each channel occasionally. Using an FFT here would give wrong or meaningless results.

4.3 The Fix: Lomb-Scargle Periodogram

In one sentence: the Lomb-Scargle method is a mathematical technique specifically built to find repeating (periodic) patterns in data **even when the samples were taken at irregular, uneven time gaps** — exactly our situation.

Simple analogy: Imagine trying to figure out how often a friend checks their phone, but you can only glance at them at random moments throughout the day (not every minute). A normal, evenly-timed analysis tool would be useless here. Lomb-Scargle is built for exactly this kind of “glancing randomly” data — it can still figure out “oh, they seem to check their phone roughly every 12 minutes” even from your scattered, irregular observations.

Once we estimate the period (and roughly when the next pulse is due), our scheduler can deliberately schedule a dwell on that channel right before the predicted burst — turning a lucky guess into a **predictive dwell**, directly improving our “Average Intercept Time Error” metric (explained in Part 6).

PART 5 — THE TRAINING GROUND: THE SIMULATOR

We cannot train or test our software on a real battlefield. So we build a realistic, simplified digital “practice world” — the **RF Environment Simulator**. This is the single most important piece of engineering in the whole project, because **every claim of success we make is only as trustworthy as this simulator is honest**.

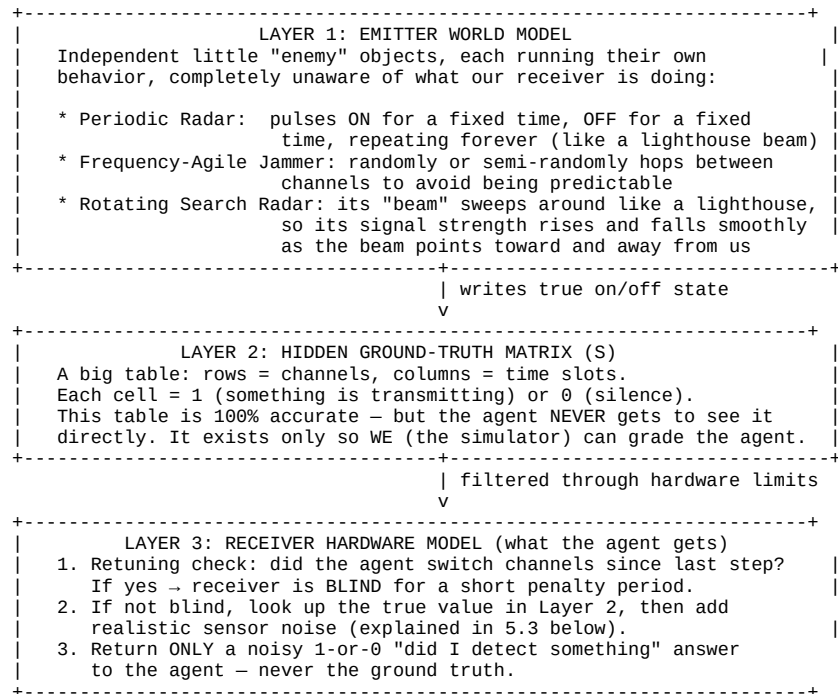
5.1 What The Simulator Is (and Is NOT)

It is NOT a physics-level simulation of actual radio waveforms (that would require simulating billions of samples per second and is computationally far too expensive for training an RL agent quickly).

It IS an abstracted scoreboard: a hidden grid that knows exactly where and when signals exist, plus a realistic model of what our receiver hardware can and cannot perceive of that grid.

Analogy: think of a video game engine. The game world (enemy emitters) runs on its own, completely unaware of and unaffected by what the player (our receiver) is currently looking at. The player only ever sees a small, filtered window into that world through the game camera.

5.2 The Three Layers of the Simulator



5.3 Sensor Noise — Why the Receiver Isn't Perfect

Real receivers sometimes **miss a real signal** (because it was weak, or noise interfered) and sometimes **falsely think they detected something when nothing was there** (because random background noise crossed the detection threshold). We model this with two probabilities:

- **Probability of Detection (P_d):** if a signal really is present, what is the chance we correctly detect it? This depends on how strong the signal is relative to background noise (the **Signal-to-Noise Ratio, or SNR**) — stronger signal, higher P_d .
- **Probability of False Alarm (P_{fa}):** if nothing is really there, what is the chance random noise fools us into thinking we detected something?

We calculate these using **Albersheim’s equation** — a well-known, textbook-standard formula that relates required SNR to a target P_d and P_fa. This is important because it’s **defensible** — if a judge asks “where does this noise model come from,” the answer is a cited, respected formula, not something we made up.

Worked example

Say for a particular channel and signal strength, our model gives $P_d = 0.85$ and $P_{fa} = 0.05$.

- If the true state is “signal present” ($s = 1$), there’s an 85% chance our receiver correctly says “detected,” and a 15% chance it misses it entirely (a **missed detection**).
- If the true state is “silence” ($s = 0$), there’s a 5% chance our receiver incorrectly says “detected” (a **false alarm**), and 95% chance it correctly reports silence.

This randomness is exactly why we can’t just trust one single observation — the agent needs to build up confidence over multiple observations, which is exactly what the “belief updater” in our closed loop (Part 2.1) does.

5.4 Retuning Latency — Why Switching Costs You

Every real receiver has physical electronics (a **local oscillator**) that need a small amount of time to “settle” onto a new frequency whenever you switch channels. During this settling window (τ_{switch}), the receiver **cannot detect anything at all** — it is completely blind, regardless of what’s actually happening on the new channel.

This is why our reward function (Part 2.3) includes a small penalty for switching channels — in real hardware, switching isn’t free, and our simulator honestly models that cost so our agent learns to avoid switching pointlessly.

5.5 Important, Honest Limitation to State Out Loud

Our emitters in this simulator are **open-loop** — they don’t change their behavior in reaction to being detected. In real combat, some advanced emitters might notice they’re being intercepted and change tactics. Our current version does not model that. **This should be stated up front as a deliberate scoping decision**, not hidden — judges respect a team that knows and states its own boundaries far more than a team that gets caught by surprise when asked about it.

PART 6 — HOW WE PROVE IT WORKS: FIGURES OF MERIT

“Figures of merit” just means: the specific numbers we use to measure and prove performance. Below is every metric, explained plainly, with a worked numeric example for each.

1. Probability of Detection (P_d)

Meaning: out of all the times we were actually listening to a channel that truly had a signal, what fraction did we correctly catch?

$$P_d = (\text{Correct detections}) / (\text{Total dwells on truly-active channels})$$

Example: We dwelt on genuinely active channels 40 times total across the mission. We correctly detected the signal 32 of those times. $P_d = 32 / 40 = 0.80 \rightarrow$ we caught 80% of the real signals we happened to be listening to.

2. Probability of False Alarm (P_{fa})

Meaning: out of all the times we listened to a channel that was truly silent, what fraction did we wrongly think had a signal?

$P_{fa} = (\text{False detections}) / (\text{Total dwells on truly-silent channels})$

Example: We dwelt on truly silent channels 200 times. We wrongly reported a detection 10 of those times. $P_{fa} = 10 / 200 = 0.05 \rightarrow 5\%$ false alarm rate.

3. Interception Ratio / Probability of Intercept (POI)

Meaning: out of ALL the signal bursts that ever happened during the whole mission (whether we were listening to that channel or not), what fraction did we successfully catch?

$POI = (\text{Signal bursts we successfully caught}) / (\text{Total signal bursts that occurred anywhere})$

Example: Across the whole mission, 500 separate signal bursts occurred in total (across all channels). We successfully caught 275 of them. $POI = 275 / 500 = 0.55 \rightarrow$ we intercepted 55% of everything that happened, even though we can only watch one channel at a time. **This is usually the single most important headline number** — it captures the whole “one straw, many rooms” challenge in one figure.

4. Average Intercept Rate

Meaning: how many successful intercepts do we get, per unit of time?

$\text{Rate} = (\text{Total intercepts}) / (\text{Total mission time elapsed})$

Example: 275 intercepts over a 550-second mission. $\text{Rate} = 275 / 550 = 0.5$ intercepts per second.

5. Percentage of Correct Predictions

Meaning: specifically for our periodicity predictor (Part 4) — when we predicted a channel would be active at a certain time slot, how often were we right?

$\text{Accuracy} = (\text{Correct forecasts}) / (\text{Total forecasts made}) \times 100$

Example: We made 60 timing predictions total during the mission. 48 of them turned out correct. $\text{Accuracy} = 48/60 \times 100 = 80\%$.

6. Average Intercept Time Error

Meaning: for periodic signals we tried to predict, on average, how far off in time was our prediction from the actual burst?

$\text{Error} = (1/K) \times \sum |\text{Predicted time} - \text{Actual time}|$ (K = number of predictions made)

Example: Across 5 predictions, our timing errors (in milliseconds) were: 2, 4, 1, 3, 5. Average = $(2+4+1+3+5)/5 = 3$ **ms average error**. Lower is better — it means our predictive dwells are landing very close to the real burst.

7. Learning Convergence (Average Reward Over Training)

Meaning: as training progresses (more episodes played), is the total reward the agent earns trending upward and then leveling off (a sign it's genuinely learning and has found a stable good strategy)? We usually show this as a line graph — reward on the y-axis, training episode number on the x-axis.

PART 7 — COMPARISON TABLE: OLD WAY vs OUR WAY

Feature	Round-Robin (classic)	Priority-Weighted Sweep	Our Smart ML Scheduler
How decisions are made	Fixed order, decided before mission	Static weights from old intel	Live, based on real-time observations
Reacts to empty channels	No — keeps re-checking them anyway	No — over-checks “priority” channels even if quiet now	Yes — deprioritizes channels that keep coming up empty
Finds brand-new/unexpected threats	Only by pure luck of the sweep order	Poorly — biased toward pre-listed threats only	Well — has a built-in reward bonus specifically for discovering new signals
Can predict WHEN a signal will reappear	No	No	Yes — via the periodicity estimator (Part 4)
Vulnerable to “blind zone” bad luck	Highly vulnerable	Moderately vulnerable	Much less vulnerable — adapts dynamically

PART 8 — WHY WE USE MACHINE LEARNING AT ALL (NOT JUST MATH FORMULAS)

There already exist classical, closed-form mathematical formulas for optimal search (for example, the **Self and Smith search equations**, a well-known result in EW literature). A fair question: why not just use those instead of machine learning?

The answer is right there in the problem statement’s title: *“in the absence of prior reliable intelligence of emitters and their operating characteristics.”*

Classical closed-form formulas like Self and Smith’s equations only give you the mathematically optimal answer **if you already know the emitter’s exact parameters in advance** (its pulse rate, rotation period, frequency, etc.) — information we are explicitly told we don’t have.

Our machine learning approach instead **learns those parameters live, from scratch, during the mission itself**, purely from its own hits and misses. This is the whole point of the project: build a system that gets smart about *unknown, never-seen-before* emitters, not just ones we already had a textbook answer for.

PART 9 — DEFENDING THE DESIGN: FULL Q&A PREP

Read through every question below and be ready to answer it in your own words, out loud, without reading from a script.

Q1: Why use Machine Learning instead of a fixed mathematical formula? A: Fixed formulas (like Self and Smith's search equations) only work when we already know the emitter's exact behavior in advance. This problem statement is specifically about situations where we do NOT have that prior intelligence. Our ML approach learns emitter behavior live, from real-time hits and misses, so it can adapt to threats we've never seen before.

Q2: What stops your system from just camping on the first signal it finds, and ignoring everything else? A: Two things. First, our reward function gives a much bigger reward for discovering a brand-new, never-seen signal than for re-checking one we already know about — this actively encourages exploring. Second, in the Whittle Index approach specifically, channels we haven't checked in a while accumulate rising "urgency," which naturally pulls the agent back toward under-explored channels over time.

Q3: Deep learning models are usually slow to train. What happens right at the very start, before the model has learned anything ("cold start")? A: We use a layered approach. In the earliest phase, when we have almost no data, the system relies on simpler methods — UCB or the Whittle Index heuristic — both of which are mathematically designed to explore sensibly even with zero prior data. As real observations accumulate, more advanced predictive components (like the periodicity estimator, or a trained neural network if we build one) take over and improve performance further.

Q4: Why call this a "Restless" Bandit instead of a normal Multi-Armed Bandit? A: In a normal bandit, an option you're not currently choosing stays frozen — unchanged until you pick it again. In our case, an enemy emitter can turn on, turn off, or hop to a new frequency completely independent of whether we're currently watching that channel or not. That constant background change is exactly what "restless" means, and it's why the plain bandit math from a casino slot-machine problem needs to be extended to handle our case.

Q5: How is this different from just building a bigger, more expensive multi-channel receiver that watches everything at once? A: That's the traditional hardware-heavy solution — banks of parallel receivers, which are large, heavy, power-hungry, and expensive. Our approach is a pure software upgrade: it can run on existing single-channel or narrowband receivers already deployed in the field, on platforms like small UAVs or lightweight ground vehicles where size, weight, power, and cost (SWaP-C) are tightly constrained. We get smarter *listening*, not more expensive *hardware*.

Q6: What are the actual limitations of your system, honestly? A: Our simulated emitters are open-loop — they don't react or change behavior in response to detecting that they're being watched. Real advanced adversaries might. Our detection noise model is also a simplified single-channel Bernoulli approximation using Albersheim's equation, not a full physical waveform simulation. We scope the project this way deliberately, because it lets us focus our time on the scheduling and prediction algorithms, which is the actual core of what this problem statement is asking for.

Q7: What if two enemy emitters share the exact same period, or if the pattern is truly random with no period at all? A: For genuinely random, non-periodic emitters, our periodicity estimator (Lomb-Scargle) simply won't find a strong periodic signal, and the scheduler falls back on the bandit/Whittle-index approach, which doesn't rely on periodicity at all — it just tracks general “how active has this channel been” statistics. For two emitters sharing a period on different channels, our system tracks each channel's history independently, so it can still form separate, correct predictions for each.

Q8: How do you prove this isn't just overfitting to one lucky test scenario? A: We deliberately evaluate our trained scheduler on a held-out scenario — a different mix of emitters, hop rates, and randomness than what it was trained on — and report performance there, not just on the training scenario. This shows the learned strategy generalizes, rather than having simply memorized one specific setup.

PART 10 — GLOSSARY (QUICK REFERENCE)

Term	Plain meaning
ESM	Electronic Support Measures — the passive “listening only” branch of EW we are building for
Emitter	Any device (radar, radio, jammer) that transmits a signal we're trying to detect
Channel / Band	One narrow slice of the total frequency spectrum
Instantaneous Bandwidth	How wide a slice of spectrum our receiver hardware can look at in one moment
Dwell	One discrete unit of time where we're listening to a chosen channel
Tuning Latency (τ_{switch})	The delay/blindness that happens when switching channels
Open-loop	A strategy fixed in advance, that doesn't adapt to what actually happens
Closed-loop	A strategy that updates itself based on real-time observations
Restless Bandit	A decision problem where unwatched options keep changing on their own
POMDP	Partially Observable Markov Decision Process — we never see the full true state, only a partial glimpse
State / Action / Reward	The three core ingredients of Reinforcement Learning: what we know, what we choose, how we're scored

Term	Plain meaning
UCB	A “smart” scheduling algorithm that balances known-good options against uncertain, under-trying options
Thompson Sampling	A “smart” scheduling algorithm that uses random sampling from belief distributions to balance explore/exploit
Whittle Index	An index number per channel representing urgency-to-check-now; purpose-built for restless bandits
DQN	Deep Q-Network — a neural-network-based RL method; our optional stretch goal
Lomb-Scargle Periodogram	A math tool for finding repeating patterns in unevenly-spaced (irregular) time data
Pd (Probability of Detection)	Chance we correctly detect a real signal when it’s there
Pfa (Probability of False Alarm)	Chance we wrongly think we detected something when nothing was there
POI (Probability of Intercept)	Fraction of ALL real signals in the whole mission that we actually caught
SWaP-C	Size, Weight, Power, and Cost — the real-world constraints our software-only solution helps reduce

PART 11 — THE 30-SECOND SUMMARY (MEMORIZE THIS)

“Old-style electronic warfare receivers sweep the radio spectrum in a fixed, pre-planned pattern, so they waste time re-checking known-quiet frequencies and give no special attention to new or evasive threats. We treat this as a restless multi-armed bandit problem: every frequency channel’s activity changes over time whether or not we’re watching it, so our receiver has to constantly balance exploring for new signals against exploiting channels it already knows are active. We built a simulated RF environment with realistic emitter behavior — periodic radar, rotating-beam radar, and frequency-hopping jammers — plus a genuine sensor noise and channel-switching-delay model, and we compare simple fixed-pattern baselines against smarter bandit-based and Whittle-index schedulers, along with a periodicity predictor that anticipates when a repeating signal will next appear. Across every key metric — probability of intercept, detection accuracy, and timing accuracy — our adaptive system beats the old fixed-pattern approach, with the biggest improvement against exactly the threats that old-style scanning handles worst: periodic and frequency-agile emitters.”

End of guide. Everything a teammate needs to understand the problem, the theory, the algorithms, the simulator, and how to defend the project verbally is contained above. Build and code specifics live in the separate build document — this file is theory-only, by design.